Arcade Architecture Manual

This manual explains procedure flow and how major components collaborate.

1. Startup Procedure

1. Main receives the default library path as argument.
2. It checks that the given .so exposes getLibraryType and is a renderer.
3. It builds game and renderer path lists.
4. It creates Arc::Core and calls run().

2. Core Main Loop Procedure

In Core::run:

1. Poll events from current renderer.
2. Forward key events to current game via handleKey.
3. Call game update.
4. Ask renderer to display shared items.
5. If game is over, save score information and return to menu.

Recent timing design in this project:

- Core uses a steady_clock based frame pacing target to reduce machine-dependent speed differences.

3. Shared Data Procedure

The map of items is the central state bus between game and renderer.

- Games create, modify, and remove items.
- Renderers draw items according to drawOrder and visibility.
- Menu uses buttonAction callbacks for UI actions.

This design reduces direct coupling between game logic and rendering backend.

4. Menu Procedure

Menu acts as a special game implementing IGame.

1. Username flow happens first.
2. User selects game and renderer.
3. Start button calls Core::launchGame.

Core remains the only owner of active plugin instances.

5. Dynamic Loading Procedure

DLLoader<T> wraps:

- dlopen: open shared library.
- dlsym: resolve entryPoint symbol.
- factory call: instantiate IGame or IRenderer implementation.

Failure at any stage throws an exception that Core catches at launch time.

6. Game Lifecycle Procedure

For each selected game:

1. Core clears map and prepares baseline background item.
2. Core loads game plugin and creates game instance using shared item map.
3. Game reacts to input and updates map each frame.
4. Game returns isGameOver true when control should return to menu.

5. Core aggregates scores and re-instantiates menu.

7. Renderer Switching Procedure

When user changes renderer and starts a game:

1. Core compares requested renderer with current renderer.
2. If different, current renderer is unloaded and new one is loaded.
3. Window is recreated by the new backend.
4. Existing game launch continues on the new renderer.

Important renderer consistency rules:

- Keep the same Arc::Event mapping semantics.
- Respect hidden items when size is non-positive.
- Draw text only when intended by item content policy.

8. Score Persistence Procedure

1. Scores are loaded from Scores/*.txt at startup.
2. During play, Core updates in-memory score tables.
3. On exit, Core writes best values to user score file.

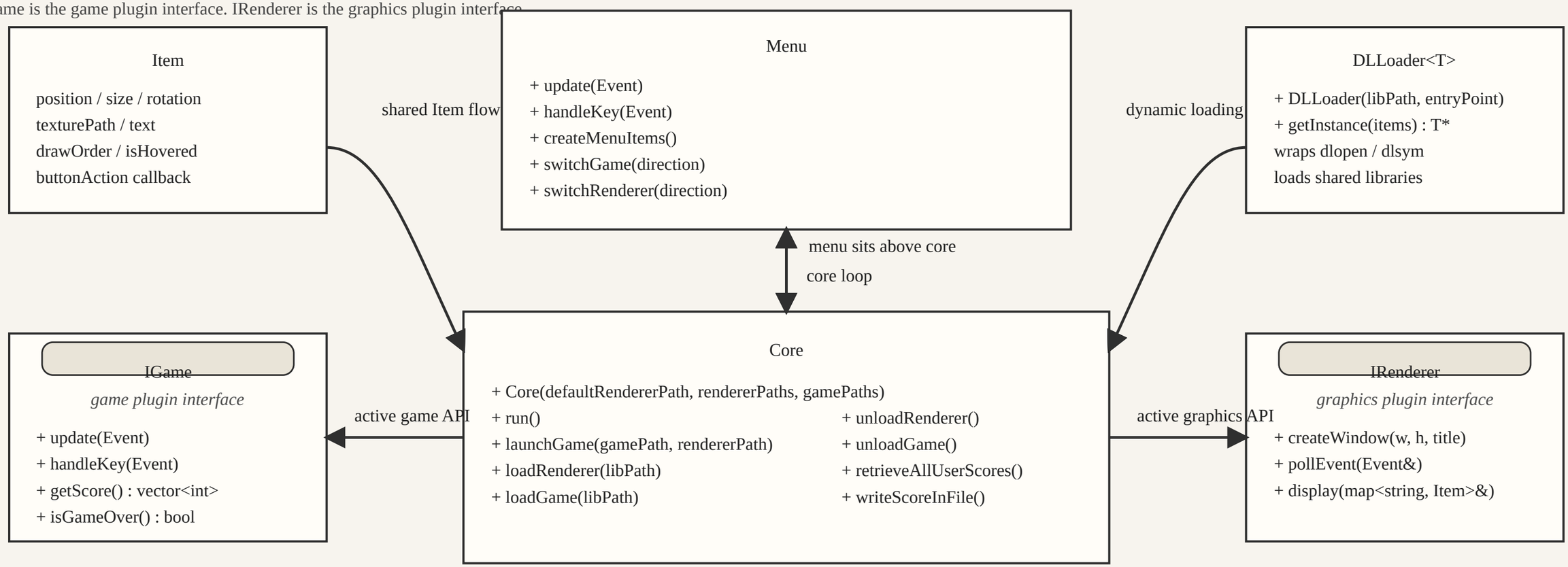
Tron has two independent score channels:

- Tron_Blue_Player
- Tron_Orange_Player

9. Extension Procedure Summary

To add a new game or renderer safely:

1. Implement interface contract exactly.
2. Export entryPoint, getLibraryType, getName.
3. Build as shared object in lib.
4. Confirm event mapping and visibility behavior.
5. Validate menu launch, play loop, and return-to-menu flow.

Arcade Class Diagram

Core owns the active IGame and IRenderer. Menu is shown above Core, with Item and DLLoader in the free spaces.
IGame is the game plugin interface. IRenderer is the graphics plugin interface.